

Tecnológico de Costa Rica
Escuela de Ingeniería Electrónica
Programa de Licenciatura en Ingeniería Electrónica



**Módulo Hardware para Estimación de Consumo Energético por
Categoría de Instrucción en Arquitecturas RISC-V**

Anteproyecto de Trabajo Final de Graduación

Jeremy Jesús Soto Chacón

Profesor Asesor: Luis G. León Vega

Campus Tecnológico Local San Carlos

San Carlos, Costa Rica, 2026

Declaro que el presente Proyecto de Graduación ha sido realizado enteramente por mi persona, utilizando y aplicando literatura referente al tema e introduciendo conocimientos propios.

En los casos en que he utilizado bibliografía he procedido a indicar las fuentes mediante las respectivas citas bibliográficas. En consecuencia, asumo la responsabilidad total por el trabajo de graduación realizado y por el contenido del correspondiente informe final.

Jeremy Jesús Soto Chacón

San Carlos, 29 de mayo de 2026

Céd: 2-0833-0519

Resumen

El crecimiento del IoT ha impulsado una demanda creciente de dispositivos que operan con presupuestos de potencia muy limitados, donde la eficiencia del software es tan determinante como la del hardware. Sin embargo, los procesadores RISC-V de bajo consumo orientados a estas aplicaciones no ofrecen ningún mecanismo que permita al firmware relacionar su ejecución con una estimación de energía, obligando a los desarrolladores a recurrir a equipos externos para caracterizar el comportamiento de sus aplicaciones.

Este trabajo propone integrar directamente en el procesador CV32E40P, sobre la plataforma PULPissimo, un módulo hardware que clasifica las instrucciones ejecutadas y mantiene contadores accesibles desde el propio firmware, sin requerir sistema operativo. La estimación se apoya en un modelo por categoría de instrucción cuyos coeficientes se caracterizan experimentalmente mediante un circuito de medición de potencia dedicado para la plataforma FPGA. La instrumentación externa se utiliza para caracterizar y validar el modelo; una vez obtenidos los coeficientes, el sistema puede estimar el consumo asociado a la ejecución del firmware a partir de los contadores internos, sin repetir una medición eléctrica completa para cada análisis. Como hipótesis de trabajo, se plantea alcanzar un error relativo medio inferior al 10 % respecto a la medición eléctrica de referencia.

Palabras clave: RISC-V, CV32E40P, consumo energético, clasificador de instrucciones, FPGA, PULPissimo, bare-metal, medición de potencia, CSR

Abstract

The growth of the Internet of Things has driven increasing demand for devices that operate under strict power budgets, where software efficiency is as decisive as hardware efficiency. However, low-power RISC-V processors targeting these applications provide no mechanism for firmware to relate its execution to an energy estimate, forcing developers to rely on external laboratory equipment to characterize the behavior of their applications.

This work proposes integrating directly into the CV32E40P processor, on the PULPissimo platform, a hardware module that classifies executed instructions and maintains counters accessible from firmware itself, without requiring an operating system. The estimation relies on an instruction-category model whose coefficients are experimentally characterized using external power measurement instrumentation for the FPGA platform. This instrumentation is required for characterization and validation; once the coefficients are obtained, the system can estimate the energy consumption associated with firmware execution from the internal counters without repeating a full electrical measurement for every analysis. The working hypothesis targets a mean relative error below 10 % with respect to the reference electrical measurement.

Keywords: RISC-V, CV32E40P, energy consumption, instruction classifier, FPGA, PULPissimo, bare-metal, power measurement, CSR

Dedicatoria

Agradecimientos

Índice general

Índice de figuras	III
Índice de tablas	IV
Glosario	V
1. Introducción	1
1.1. Planteamiento del problema	2
1.1.1. Estado actual	2
1.1.2. Definición del problema	2
1.2. Estructura del documento	3
2. Marco teórico	4
2.1. Plataforma hardware	4
2.1.1. Arquitectura RISC-V	4
2.1.2. Registros de Control y Estado	5
2.1.3. Procesador CV32E40P	5
2.1.4. SoC PULPissimo	6
2.1.5. FPGA Nexys A7-100T	7
2.2. Fundamentos de análisis energético	8
2.2.1. Modelo de estimación por instrucción	8
2.3. Diseño RTL y síntesis FPGA	10
2.4. Fundamentos de medición eléctrica	11
2.5. Estado del arte	12
3. Solución propuesta	13
3.1. Medición de potencia	13
3.1.1. Módulo de sensado de corriente	14
3.1.2. Adquisición y promediado	15
3.2. Módulo clasificador de instrucciones	17
3.2.1. Concepto de clasificación	17
3.2.2. Contadores e interfaz CSR	18
3.2.3. Integración en el procesador	19
3.2.4. Carga del firmware vía JTAG	19
3.3. Caracterización y estimación	21

3.3.1. Software de análisis y operación	21
4. Hipótesis y Objetivos	23
4.1. Hipótesis	23
4.2. Objetivo general	23
4.3. Objetivos específicos	24
4.4. Tareas asociadas a los objetivos específicos	24
4.5. Alcances y limitaciones	25
5. Experimento preliminar	27
5.1. Objetivo del experimento	27
5.2. Configuración de la prueba	27
5.3. Rutinas de bucles dominados por categoría	28
5.4. Datos a recolectar	29
5.5. Criterio de viabilidad	29
6. Cronograma	31
Bibliografía	33

Índice de figuras

2.1. Formatos base de instrucción RISC-V RV32I [13].	4
2.2. Arquitectura interna del procesador CV32E40P [11].	6
2.3. Arquitectura de bloques del SoC PULPissimo [19].	7
3.1. Arquitectura general del sistema propuesto.	13
3.2. Módulo de sensado basado en INA240 con resistor de shunt integrado. . . .	14
3.3. Ubicación del resistor de sensado en la línea de alimentación de la plataforma.	15
3.4. Cadena de adquisición de potencia desde el resistor de sensado hasta el sistema ESP32.	16
3.5. Integración conceptual del módulo clasificador como observador del pipeline del CV32E40P.	19
3.6. Cadena de comunicación con el CV32E40P.	20
3.7. Módulo FT232H utilizado como interfaz USB-JTAG.	20

Índice de tablas

2.1. Instrucciones RISC-V para acceso a CSRs [13].	5
3.1. Categoría de instrucción y origen de la señal usada para clasificarla en el pipeline del CV32E40P.	18
5.1. Rutinas propuestas para el experimento preliminar.	29
6.1. Cronograma general de ejecución del proyecto.	32

Glosario

- Bare-metal** Modo de ejecución en que el software corre directamente sobre el hardware sin sistema operativo intermedio. En este trabajo se refiere al firmware que se ejecuta sobre el CV32E40P dentro del SoC PULPissimo, accediendo a los periféricos y registros CSR sin ninguna capa de abstracción de sistema operativo.
- APU** Unidad de procesamiento auxiliar (*Auxiliary Processing Unit*). Interfaz del CV32E40P que enlaza el núcleo con coprocesadores externos como la FPU.
- BRAM** Memoria embebida en bloque (*Block RAM*). Recurso de memoria síncrona integrado en las FPGA, organizado en bloques de capacidad fija.
- CV32E40P** Procesador RISC-V de 32 bits desarrollado por el OpenHW Group como evolución del núcleo RI5CY del equipo PULP. Implementa el ISA RV32IMFC con las extensiones Xpulp y constituye el núcleo central del SoC utilizado en este trabajo.
- GDB** Depurador usado para cargar firmware, controlar la ejecución y leer registros del procesador a través de OpenOCD.
- HPC** Cómputo de alto rendimiento (*High-Performance Computing*). Categoría de plataformas con sistema operativo y contadores de hardware expuestos para telemetría energética estandarizada.
- IoT** Internet de las cosas (*Internet of Things*). Ecosistema de dispositivos embebidos conectados que operan con presupuestos de potencia limitados.
- ISA** Arquitectura del conjunto de instrucciones (*Instruction Set Architecture*). Define la interfaz visible entre el software y el procesador.
- JTAG** Interfaz estándar IEEE 1149.1 usada para prueba y depuración de circuitos mediante señales serie como TCK, TMS, TDI y TDO.
- LUT** Tabla de búsqueda (*Look-Up Table*). Bloque combinacional básico de las FPGA, configurable para implementar cualquier función lógica de pocas entradas.
- MPSSE** Motor síncrono multiprotocolo de FTDI (*Multi-Protocol Synchronous Serial Engine*) usado para generar señales JTAG desde el FT232H.
- OpenOCD** Herramienta de depuración que controla el acceso JTAG y expone una interfaz para que GDB pueda cargar programas y leer registros del procesador.

SIMD Una instrucción, múltiples datos (*Single Instruction Multiple Data*). Estilo de ejecución vectorial en el que una misma instrucción procesa varios operandos empaquetados en un mismo registro.

SoC Sistema en un chip (*System-on-Chip*). Integración de procesador, memoria, buses y periféricos dentro de una misma plataforma.

TCDM Memoria de datos estrechamente acoplada (*Tightly Coupled Data Memory*). En PULPissimo se refiere al acceso directo del procesador a la SRAM interna.

Capítulo 1

Introducción

El crecimiento acelerado del IoT ha posicionado a los sistemas embebidos de ultrabaja potencia como una pieza central de la infraestructura tecnológica moderna. En 2025 se estimó que el número de dispositivos IoT conectados superaba los 21 000 millones a nivel global [15], la mayoría de los cuales operan alimentados por baterías de tamaño reducido o mediante técnicas de recolección de energía del entorno [1]. En este escenario, el presupuesto de potencia de un nodo típico se sitúa en el orden de los milivatios: el procesador, la radio, los sensores y la memoria compiten por ese margen mínimo, y la falta de eficiencia en el firmware se traduce directamente en una reducción de la vida útil del dispositivo.

Para responder a esta demanda, la comunidad de arquitecturas de procesadores ha convergido en torno a RISC-V, un conjunto de instrucciones abierto, modular y libre de regalías que permite diseñar núcleos altamente especializados para aplicaciones de bajo consumo. Dentro de este ecosistema, el grupo de investigación PULP de ETH Zürich ha desarrollado la plataforma PULPissimo [19], un SoC orientado a sistemas embebidos que integra el procesador CV32E40P [11] como su componente central de cómputo.

El CV32E40P implementa el ISA RISC-V RV32IMFCXpulp de 32 bits e incorpora extensiones vectoriales SIMD y de punto flotante que lo hacen especialmente adecuado para cargas de procesamiento de señales, control y aprendizaje automático en el borde de la red [11]. A pesar de sus capacidades, el núcleo carece de mecanismos de inspección energética: no expone contadores de potencia, no implementa una interfaz interna de medición energética y no ofrece registros dedicados que permitan obtener un perfil de ejecución útil para estimar el consumo energético [9]. Esta ausencia obliga a recurrir a instrumentación física externa para relacionar una ejecución concreta con su consumo energético. Dicha instrumentación es necesaria para caracterizar y validar modelos de consumo, pero su uso como único mecanismo de análisis dificulta la comparación repetida de regiones de código durante el desarrollo de firmware.

El modelado de consumo energético a nivel de instrucción es una técnica consolidada que puede cerrar esa brecha. Propuesta originalmente por Tiwari *et al.* [20] para procesadores Intel 486 y SPARC, se basa en la observación de que cada tipo de instrucción activa

un subconjunto distinto de las unidades funcionales del procesador, generando un perfil de consumo característico y medible. Extendida recientemente a la arquitectura RISC-V por Fang [3], esta metodología permite estimar la potencia de un programa como la suma ponderada del número de instrucciones ejecutadas por categoría. Sin embargo, la implementación clásica de esta metodología en la literatura opera de forma *off-chip* y *post-mortem*: el análisis se realiza sobre el binario ya generado, fuera del procesador.

Este trabajo propone integrar ese modelo en el hardware del CV32E40P mediante contadores accesibles por CSR. Durante la ejecución bare-metal sobre PULPissimo, el firmware ejecuta la carga de prueba y el procesador acumula el número de instrucciones retiradas por categoría. Al finalizar la ejecución o una ventana de medición, los registros del clasificador se leen desde la computadora anfitriona y se combinan con coeficientes caracterizados previamente mediante un circuito de medición de potencia. La instrumentación externa no se elimina del flujo experimental: se usa para obtener la referencia física y validar el modelo, mientras que los contadores internos permiten repetir el análisis sin ejecutar una medición eléctrica completa para cada escenario posterior.

1.1. Planteamiento del problema

1.1.1. Estado actual

La estimación de consumo energético depende de información que el procesador no siempre expone de forma directa. En plataformas con sistema operativo, parte de esa información puede obtenerse mediante infraestructura de monitoreo y herramientas externas. En una ejecución bare-metal, como la de PULPissimo sobre el CV32E40P, esa capa no existe: el firmware se ejecuta directamente sobre el hardware y el procesador no entrega por sí mismo conteos de instrucciones retiradas por categoría.

Por ello, el análisis energético de esta plataforma depende de mediciones eléctricas externas o de análisis posterior del programa ejecutado. Ambos enfoques son útiles, pero no generan desde el propio procesador los datos necesarios para asociar ejecución y consumo estimado dentro del flujo de prueba.

1.1.2. Definición del problema

El problema que aborda este trabajo puede enunciarse de forma precisa: el procesador CV32E40P, ejecutando firmware en bare-metal sobre el SoC PULPissimo sintetizado en una FPGA, no dispone de un mecanismo interno que genere un histograma completo de instrucciones retiradas por categoría ni que permita extraer esa información desde el flujo de prueba para estimar el consumo energético observado en la plataforma.

Como consecuencia, un desarrollador de firmware que necesite optimizar el consumo de su programa debe apoyarse en instrumentación eléctrica externa para cada escenario que

desea estudiar. Esta instrumentación es útil como referencia física, pero no entrega una señal interna que pueda leerse desde el flujo de prueba para comparar regiones de código o automatizar análisis posteriores.

Por tanto, la brecha central no es únicamente la ausencia de una medición física de potencia, sino la falta de un mecanismo interno que relacione la ejecución del firmware con una estimación de consumo obtenible a partir de las instrucciones vistas durante la ejecución.

1.2. Estructura del documento

En el **capítulo 2** se presenta el marco teórico, que incluye la plataforma hardware, los fundamentos de análisis energético, el diseño RTL, la medición eléctrica y el estado del arte relacionado con la estimación de consumo a nivel de instrucción.

En el **capítulo 3** se describe la solución propuesta: el circuito de medición de potencia, el módulo clasificador de instrucciones, su integración en el RTL del CV32E40P, la carga y lectura mediante JTAG, y el flujo de caracterización y análisis de datos.

En el **capítulo 4** se enuncian la hipótesis de trabajo, los objetivos del proyecto, las tareas asociadas, los alcances y las limitaciones.

En el **capítulo 5** se presenta el experimento preliminar propuesto para validar el flujo de medición y la observabilidad de diferencias de potencia entre rutinas dominadas por distintas categorías de instrucción.

En el **capítulo 6** se muestra el cronograma de ejecución del proyecto mediante un diagrama de Gantt.

Capítulo 2

Marco teórico

2.1. Plataforma hardware

2.1.1. Arquitectura RISC-V

RISC-V es un ISA abierto, modular y libre de regalías desarrollado en la Universidad de California, Berkeley [13]. La especificación base RV32I define 40 instrucciones de 32 bits distribuidas en seis formatos según la disposición de sus campos. El campo `opcode` identifica el formato; `funct3` y `funct7` discriminan variantes dentro del mismo opcode. La Figura 2.1 muestra los seis formatos base y cómo están estructurados. Además, dado que la arquitectura es modular, sobre la base RV32I se añaden extensiones opcionales que amplían el conjunto de instrucciones sin modificar la codificación base.

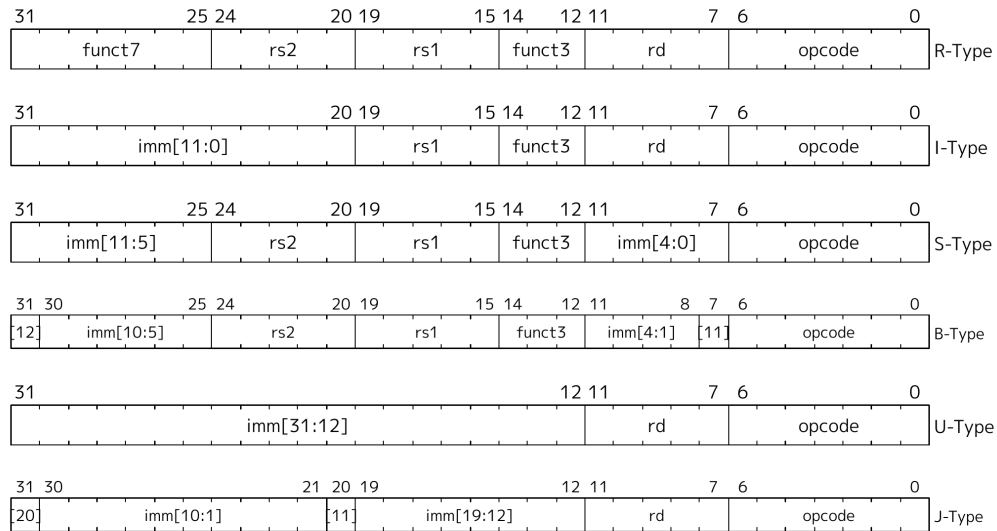


Figura 2.1: Formatos base de instrucción RISC-V RV32I [13].

2.1.2. Registros de Control y Estado

Los CSR son registros internos del procesador accesibles mediante un espacio de direcciones de 12 bits independiente de la memoria principal [13]. La especificación RISC-V define rangos estándar para contadores de rendimiento, información del procesador y control de interrupciones, y reserva un rango para CSRs personalizados en modo máquina que cada implementación puede definir libremente. El acceso se realiza mediante instrucciones atómicas, sin requerir sistema operativo ni llamadas al sistema. La Tabla 2.1 resume las instrucciones de acceso disponibles.

Tabla 2.1: Instrucciones RISC-V para acceso a CSRs [13].

Instrucción	Operación
<code>csrr rd, csr</code>	Lee <code>csr</code> y almacena el valor en <code>rd</code>
<code>csrw csr, rs1</code>	Escribe el valor de <code>rs1</code> en <code>csr</code>
<code>csrs csr, rs1</code>	Activa los bits de <code>csr</code> indicados por <code>rs1</code>
<code>csrc csr, rs1</code>	Limpia los bits de <code>csr</code> indicados por <code>rs1</code>

2.1.3. Procesador CV32E40P

El CV32E40P es un procesador RISC-V de 32 bits desarrollado por el OpenHW Group como evolución del núcleo RI5CY del equipo PULP de ETH Zürich [11]. Implementa el ISA RV32IMFC del estándar RISC-V, complementado con las extensiones vectoriales SIMD propias del ecosistema PULP o Xpulp. La Figura 2.2 muestra la arquitectura interna del núcleo, donde se distinguen las principales unidades funcionales (ALU, multiplicador, LSU y FPU) y el banco de registros que alimenta sus operandos.

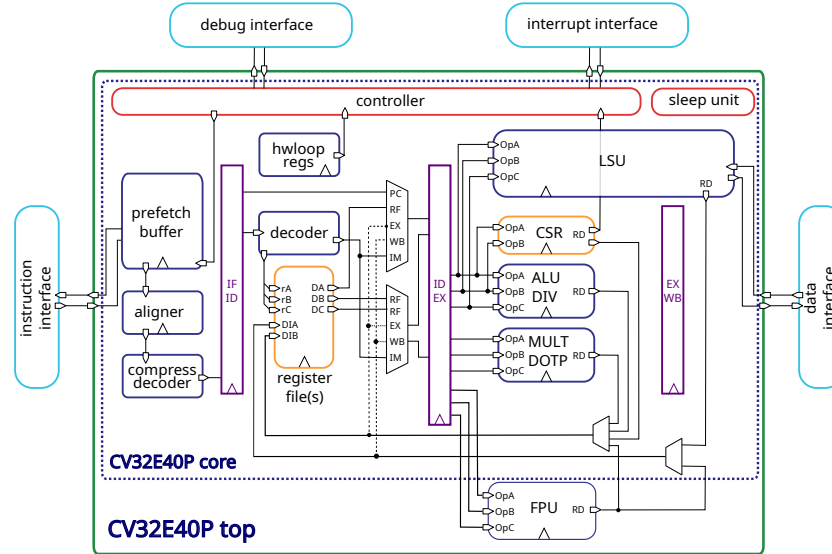


Figura 2.2: Arquitectura interna del procesador CV32E40P [11].

El núcleo utiliza un pipeline de cuatro etapas: búsqueda (IF), decodificación (ID), ejecución (EX) y escritura (WB). En condiciones ideales cada etapa procesa una instrucción por ciclo. La etapa de decodificación interpreta los campos de la instrucción RISC-V y genera las señales de control que habilitan las unidades funcionales del núcleo, como la ALU, la unidad de carga/almacenamiento, la lógica de salto y la unidad de punto flotante. Por esta razón, la etapa ID concentra información sobre el tipo de operación que será ejecutada por el procesador [11], lo que la convierte en un punto relevante para observar el comportamiento funcional del pipeline.

Adicionalmente, el CV32E40P integra soporte de depuración externa compatible con JTAG, lo que permite observar y controlar el estado del procesador durante pruebas de desarrollo [5].

2.1.4. SoC PULPissimo

PULPissimo es un SoC de un solo núcleo del equipo PULP de ETH Zürich orientado a aplicaciones de ultrabaja potencia [19]. Su arquitectura integra el CV32E40P como núcleo de cómputo central, rodeado de los bloques de soporte necesarios para operar en modo bare-metal sin sistema operativo. El acceso a memoria se realiza a través del bus TCDM, que garantiza latencia de un ciclo desde el procesador hacia la SRAM interna, eliminando la variabilidad de latencia que introducirían cachés convencionales [19].

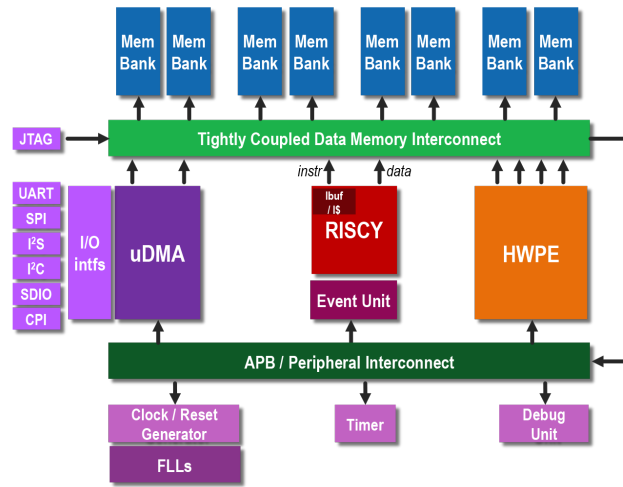


Figura 2.3: Arquitectura de bloques del SoC PULPissimo [19].

Como se observa en la Figura 2.3, el SoC incluye periféricos estándar (SPI, I2C, GPIO) accesibles mediante el bus APB, un subsistema de DMA para transferencias sin intervención del procesador, y un *fabric controller* que gestiona el arranque y la comunicación con el módulo de depuración JTAG.

2.1.5. FPGA Nexys A7-100T

Una FPGA es un circuito integrado cuya lógica puede reconfigurarse después de su fabricación mediante un archivo de configuración denominado *bitstream*. A diferencia de un ASIC, cuya función queda fija en manufactura, una FPGA permite implementar cualquier circuito digital descrito en un lenguaje de descripción de hardware (HDL) y cargarlo en minutos, lo que la convierte en la plataforma estándar para realizar prototipo de diseños RTL.

Internamente, los recursos de una FPGA se organizan en LUT que implementan lógica combinacional arbitraria, flip-flops que almacenan estado, BRAMs síncronas y bloques DSP (multiplicadores de hardware). La Nexys A7-100T utilizada en este proyecto incorpora una FPGA Artix-7 (XC7A100T-1CSG324C) con 15 850 slices lógicos (126 800 flip-flops), 4 860 Kb de Block RAM y 240 DSP slices [2], suficientes para albergar el SoC PULPissimo completo.

La implementación de un procesador sobre FPGA no reproduce el comportamiento eléctrico de un ASIC. La matriz de interconexión programable, los recursos lógicos configurables y las tensiones de operación propias de la placa modifican la capacitancia efectiva y la potencia dinámica observada. Por ello, los coeficientes energéticos deben caracterizarse para la plataforma concreta donde se ejecuta el diseño.

2.2. Fundamentos de análisis energético

La potencia consumida por un circuito digital puede separarse, de forma aproximada, en una componente estática y una componente dinámica:

$$P_{\text{total}} = P_{\text{estática}} + P_{\text{dinámica}} \quad (2.1)$$

La potencia estática agrupa el consumo presente aun cuando no hay actividad de conmutación útil, asociado principalmente a corrientes de fuga y a bloques de la plataforma que permanecen activos. En una FPGA, esta componente puede ser significativa porque el dispositivo completo, la red de reloj y la lógica de soporte permanecen alimentados durante la ejecución.

La potencia dinámica corresponde al consumo producido por la conmutación de nodos internos durante la operación del circuito. En tecnología CMOS, esta componente se aproxima mediante [20]:

$$P_{\text{dinámica}} = \alpha \cdot C_{\text{eff}} \cdot V_{DD}^2 \cdot f \quad (2.2)$$

donde α es el factor de actividad, C_{eff} la capacitancia efectiva conmutada, V_{DD} la tensión de alimentación y f la frecuencia de reloj. Para una plataforma y una frecuencia fijas, los términos V_{DD}^2 y f son constantes, de modo que la variación de potencia entre instrucciones queda determinada por α y C_{eff} . Estos dos factores dependen de qué unidades funcionales del procesador conmutan en cada ciclo: una instrucción que activa la FPU conmuta un número distinto de nodos que una que solo emplea la ALU o que una que accede al bus de memoria. En consecuencia, el consumo dinámico depende del tipo de instrucción ejecutada, y agrupar instrucciones según las unidades funcionales que activan es el principio sobre el que se construye el modelo de estimación por instrucción.

2.2.1. Modelo de estimación por instrucción

Tiwari *et al.* [20] propusieron en 1994 que el consumo de un programa puede estimarse como la suma ponderada de costos energéticos por categoría:

$$E_{\text{est}} = \sum_{i=0}^{N-1} e_i \cdot n_i \quad (2.3)$$

donde e_i es el costo energético de la categoría i en J/instrucción y n_i el número de instrucciones de esa categoría ejecutadas. Si el intervalo de ejecución tiene duración T , la potencia promedio estimada se obtiene como $\bar{P}_{\text{est}} = E_{\text{est}}/T$. Este modelo no requiere sistema operativo, su evaluación es trivial y sus coeficientes son reutilizables para cualquier programa en la misma plataforma. La diferencia entre las implementaciones existentes

radica en el método de caracterización de los coeficientes e_i , para el cual existen dos enfoques principales.

Método de bucles dominados por categoría

El método de bucles dominados por categoría se basa en ejecutar una rutina cuya actividad esté concentrada en una clase de instrucciones conocida. Esta idea fue propuesta por Tiwari [20] para medir costos energéticos por instrucción y posteriormente aplicada a RISC-V por Fang [3]. En términos del modelo de la ecuación 2.3, este método busca construir una ejecución donde una categoría i domina el histograma de instrucciones. Si se mide la potencia promedio $\bar{P}$ durante un intervalo T , la energía observada puede asociarse principalmente a esa categoría y el costo energético promedio se aproxima como:

$$e_i = \frac{\bar{P} T}{n_i} \quad (2.4)$$

El conteo n_i puede obtenerse a partir del código ensamblador, de la duración del intervalo, de la frecuencia de reloj o de contadores de ejecución. La ventaja del método es que concentra la actividad del procesador en una categoría controlada. En la práctica, una rutina de este tipo no puede estar compuesta exclusivamente por instrucciones de una sola categoría, porque la estructura del bucle requiere instrucciones auxiliares, como actualizaciones aritméticas de índices o punteros y saltos para mantener la repetición. Por ello, como criterio de diseño se busca que al menos el 95 % de las instrucciones ejecutadas pertenezcan a la categoría que se desea caracterizar, reduciendo así la influencia de esas instrucciones auxiliares sobre el coeficiente estimado. Su principal limitación es que no captura completamente las interacciones entre categorías diferentes.

Método de regresión por histograma

En enfoques basados en regresión por histogramas, se ejecuta un conjunto de cargas de trabajo diversas mientras se miden la energía consumida y el número de instrucciones ejecutadas por categoría. Para cada ejecución, la energía medida y el histograma de instrucciones deben satisfacer el modelo de la ecuación 2.3.

Repitiendo el experimento sobre M programas distintos se obtienen M ecuaciones independientes con las mismas N incógnitas $e_0, \dots, e_{N-1}$. Cuando $M > N$, el sistema tiene más ecuaciones que incógnitas y, en general, no admite una solución exacta, porque cada medición incluye ruido eléctrico y desviaciones respecto al modelo. La forma matricial del sistema es:

$$\begin{bmatrix} E_1 \\ \vdots \\ E_M \end{bmatrix} = \begin{bmatrix} n_{0,1} & \cdots & n_{N-1,1} \\ \vdots & \ddots & \vdots \\ n_{0,M} & \cdots & n_{N-1,M} \end{bmatrix} \begin{bmatrix} e_0 \\ \vdots \\ e_{N-1} \end{bmatrix} \quad (2.5)$$

que se resuelve por mínimos cuadrados: se buscan los coeficientes e_i que minimizan la suma de los cuadrados de las diferencias entre la energía medida en cada ejecución y la energía predicha por el modelo. De esta forma, los coeficientes resultantes son los que mejor explican el conjunto completo de mediciones, no los de una sola ejecución particular. Este método puede producir coeficientes más representativos para cargas mixtas, pero requiere un conjunto de programas suficientemente diverso y mediciones de potencia asociadas a cada ejecución.

La calidad del ajuste depende de que los M programas seleccionados produzcan una matriz de conteos bien condicionada. Si todos contienen una mezcla similar de instrucciones, las columnas de esa matriz se vuelven aproximadamente linealmente dependientes y el sistema queda mal condicionado, lo que amplifica el ruido de medición en la estimación de e_i . Este efecto puede diagnosticarse mediante métricas como el número de condición de la matriz de conteos o el coeficiente de determinación R^2 .

Supuestos del modelo

El modelo de la ecuación 2.3 se apoya en supuestos que acotan su precisión. El primero es que la potencia estática o de fondo puede tratarse como una contribución aproximadamente constante durante la ventana de medición. Si no se separa de la potencia dinámica, parte de ese consumo queda absorbido dentro de los coeficientes e_i .

El segundo supuesto es que el costo energético promedio de una instrucción puede asociarse a su categoría, aunque en la práctica depende parcialmente del contexto de ejecución. Efectos como el reuso de datos, el estado de buses internos, el *forwarding* o la secuencia inmediata de instrucciones pueden introducir desviaciones respecto al modelo lineal. Por esta razón, la validez de los coeficientes debe comprobarse con cargas distintas a las usadas durante la caracterización.

2.3. Diseño RTL y síntesis FPGA

SystemVerilog [6] es el lenguaje de descripción de hardware en el que está escrito el ecosistema PULP [19]. A diferencia de los lenguajes de programación convencionales, en los que cada instrucción se ejecuta una tras otra, un lenguaje de descripción de hardware enuncia qué circuitos existen y cómo están conectados; todos esos circuitos operan al mismo tiempo y avanzan al ritmo de un reloj común.

A nivel RTL, el diseño se piensa como un conjunto de registros entre los que fluyen datos a través de lógica combinacional. Esta vista permite razonar sobre el comportamiento del circuito sin entrar en el detalle físico de cada compuerta, y es el nivel de abstracción al que se desarrolla el hardware nuevo de este trabajo.

El mismo código RTL admite dos usos complementarios: la *simulación*, que permite verificar el comportamiento del diseño antes de implementarlo, y la *síntesis*, que lo traduce a

los recursos físicos de la FPGA para ejecutarlo en hardware real. Esta separación permite concentrar la verificación funcional en simulación y reservar la síntesis para las pruebas en plataforma.

2.4. Fundamentos de medición eléctrica

La potencia eléctrica consumida por un circuito digital puede determinarse a partir de la tensión de alimentación y la corriente demandada por la carga. En estado estacionario, la relación básica está dada por:

$$P = V \cdot I \quad (2.6)$$

donde V corresponde a la tensión aplicada al circuito e I a la corriente que circula por la línea de alimentación. En plataformas digitales, la medición de corriente debe realizarse sin modificar de forma significativa las condiciones eléctricas de operación, ya que variaciones en la tensión de alimentación pueden alterar la frecuencia máxima, la estabilidad del sistema o el consumo observado.

Un método común para estimar corriente consiste en insertar una resistencia de derivación (*shunt*) en serie con la carga. La caída de tensión sobre esta resistencia es proporcional a la corriente, según:

$$I = \frac{V_{R_{\text{shunt}}}}{R_{\text{shunt}}} \quad (2.7)$$

Para reducir la perturbación introducida por la medición, el valor del shunt se elige de forma que la caída de tensión sea pequeña respecto a la alimentación del sistema, pero suficientemente grande para ser medida con precisión. Esta señal suele requerir acondicionamiento analógico [16], debido a que las variaciones de tensión pueden estar en el orden de milivoltios y coexistir con ruido de conmutación producido por la actividad del circuito digital.

La resistencia de derivación puede colocarse en el lado alto de la alimentación (*high-side*) o en el lado bajo de la carga (*low-side*). La medición *high-side* conserva la referencia de tierra de la carga, pero exige medir una pequeña diferencia de tensión superpuesta a un voltaje de modo común mayor. La medición *low-side* simplifica la adquisición de la señal, aunque introduce una diferencia entre la tierra de la carga y la tierra del sistema de medición [18]. La selección entre ambas topologías depende de las restricciones eléctricas de la plataforma y del instrumento de adquisición utilizado. La aplicación concreta de estos fundamentos al circuito de medición construido para este trabajo, junto con la selección de R_{shunt} , la ganancia del amplificador y la cadena de adquisición, se detalla en la sección 3.1.

2.5. Estado del arte

El análisis de potencia a nivel de instrucción tiene su origen en el trabajo seminal de Tiwari *et al.* [20], publicado en 1994 para procesadores Intel 486 y SPARC. Demostró que el consumo de un programa puede modelarse como suma ponderada de costos por instrucción, principio generalizable a cualquier arquitectura RISC. Fang [3] extendió el modelo a un ASIC RISC-V RV32IM a 1,1 V y 40 MHz, obteniendo coeficientes de consumo por instrucción para esa implementación. Sus resultados confirman el vínculo entre tipo de instrucción y consumo, pero sus coeficientes no son transferibles directamente a una FPGA: cambian la tecnología física, la red de interconexión, la capacitancia efectiva, la tensión y la frecuencia de operación. Además, esa implementación de referencia opera *off-chip* y *post-mortem*: analiza el binario descompilado fuera del procesador, sin integración en hardware que genere conteos durante la ejecución real.

En sistemas de alto rendimiento, EfiMon [9] estima el consumo de procesos individuales sobre x86 con error del 2,2 %; una extensión sobre AMD [8] mantiene error inferior al 4,4 %. Ambos enfoques dependen de infraestructura disponible en sistemas con software de propósito general y no son portables directamente a una ejecución bare-metal.

El trabajo más próximo al enfoque propuesto es el de Jiménez Arribas *et al.* [7], que implementa contadores hardware directamente en el RTL de un procesador RISC-V sobre la misma familia de FPGA Artix-7 utilizada en este trabajo, lo que confirma la viabilidad de exponer contadores internos en esta plataforma. Sin embargo, su objetivo es la caracterización del rendimiento temporal (CPI, ciclos por evento) mediante CSRs estándar de 64 bits, sin asociar los contadores a coeficientes de potencia ni realizar mediciones eléctricas sobre la plataforma.

Las contribuciones revisadas cubren elementos individuales del problema: el modelado por categoría [20, 3], la medición eléctrica sobre RISC-V [3], la asignación de potencia con apoyo del sistema operativo [9, 8] y el conteo de eventos en hardware [7]. Sin embargo, ninguna integra estos elementos en una plataforma bare-metal con contadores internos que permitan extraer un perfil de ejecución asociable a una estimación de consumo en el flujo de análisis. Ese vacío específico es el que motiva el presente trabajo.

Capítulo 3

Solución propuesta

La estimación de consumo energético asociada a firmware bare-metal requiere combinar tres componentes que operan de forma coordinada sobre el CV32E40P: un circuito de medición eléctrica que adquiere la potencia real consumida por la plataforma, un módulo hardware integrado en el procesador que clasifica las instrucciones ejecutadas y las acumula en contadores accesibles por CSR, y una etapa de análisis de datos que relaciona ambas fuentes para obtener los coeficientes del modelo energético. La Figura 3.1 ilustra la interacción entre los tres componentes.

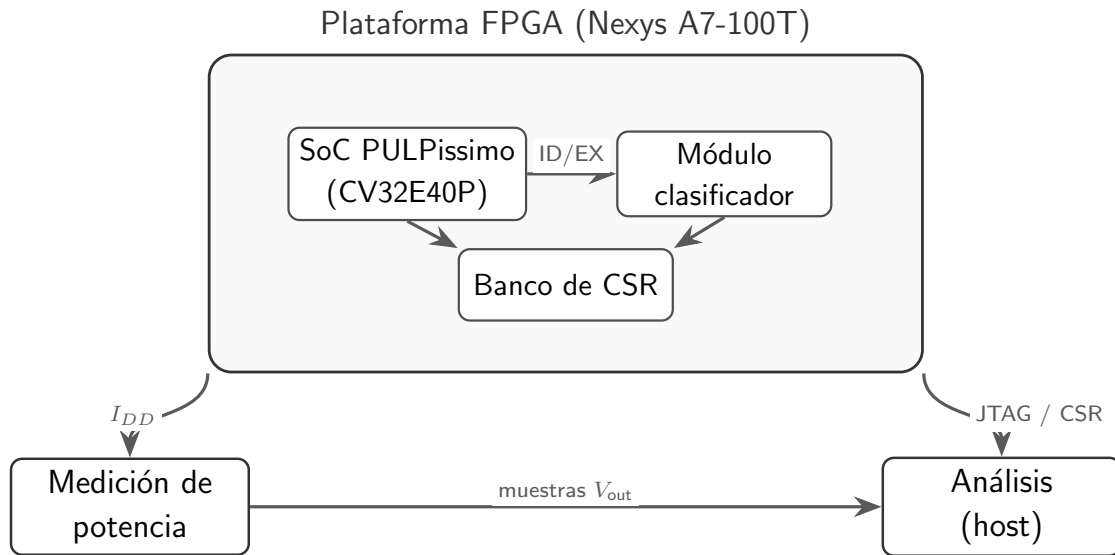


Figura 3.1: Arquitectura general del sistema propuesto.

3.1. Medición de potencia

La potencia de referencia se obtiene midiendo la corriente en la línea de alimentación de la Nexys A7-100T durante la ejecución del SoC PULPissimo. Esta medición corresponde al consumo observado en la plataforma FPGA completa bajo la configuración experimental,

no al consumo aislado del núcleo como circuito independiente. La cadena de medición consta de un módulo de sensado basado en INA240A1 con resistor de shunt integrado, un convertidor analógico-digital ADS1115 y un sistema de adquisición autónomo que registra la tensión de salida para su procesamiento posterior.

3.1.1. Módulo de sensado de corriente

La medición de corriente se realiza con un módulo conectado en serie con la línea de 5 V del jack de alimentación de la placa. Este módulo integra el amplificador INA240A1 y un resistor de shunt marcado como R100, equivalente a $R_{\text{shunt}} = 100 \text{ m}\Omega$. La corriente demandada por la carga genera sobre este resistor una caída de tensión proporcional [18]:



Figura 3.2: Módulo de sensado basado en INA240 con resistor de shunt integrado.

$$V_{R_{\text{shunt}}} = I \cdot R_{\text{shunt}} \quad (3.1)$$

El valor de R_{shunt} se eligió como compromiso entre perturbación de la alimentación y resolución de medición. Para el consumo máximo de placa reportado por la Nexys A7-100T, 5 W desde una fuente de 5 V [2], la corriente es de 1 A y la caída sobre el shunt es de 100 mV, equivalente al 2 % de V_{DD} .

La potencia instantánea se obtiene como:

$$P = V_{DD} \cdot \frac{V_{R_{\text{shunt}}}}{R_{\text{shunt}}} \quad (3.2)$$

El resistor se coloca en configuración *high-side*, en serie entre la fuente de alimentación y la carga, para medir directamente la corriente entregada por la fuente a la Nexys A7-100T [18]. En esta posición, la señal diferencial sobre el shunt queda superpuesta a la línea de 5 V, por lo que el amplificador debe medir caídas pequeñas en presencia de un voltaje de modo común cercano a la tensión de alimentación.

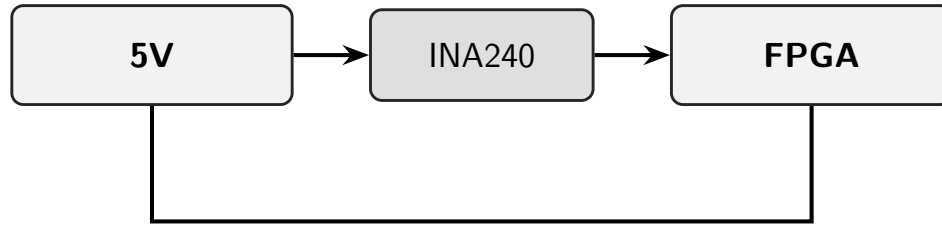


Figura 3.3: Ubicación del resistor de sensado en la línea de alimentación de la plataforma.

El INA240 [16] se utiliza porque mide caídas diferenciales pequeñas sobre resistores de sensado en configuración *high-side*. La variante INA240A1 aporta una ganancia fija de $G = 20 \text{ V/V}$, suficiente para llevar la caída producida por el shunt a un nivel adecuado para el ADS1115 sin saturar la entrada en el rango de corriente usado para dimensionar la medición.

La salida del INA240 es una tensión proporcional a la corriente consumida:

$$V_{\text{out}} = G \cdot V_{R_{\text{shunt}}} = G \cdot I \cdot R_{\text{shunt}} \quad (3.3)$$

Despejando la corriente y sustituyendo en la ecuación 3.2, la potencia se obtiene directamente de la lectura del ADC:

$$P = V_{DD} \cdot \frac{V_{\text{out}}}{G \cdot R_{\text{shunt}}} \quad (3.4)$$

Con $R_{\text{shunt}} = 100 \text{ m}\Omega$ y $G = 20 \text{ V/V}$, la tensión de salida queda dada por:

$$V_{\text{out}} = 20 \cdot I \cdot 0,1 \Omega = 2I \quad (3.5)$$

Como ejemplo, una corriente de 400 mA produciría $V_{\text{out}} = 0,8 \text{ V}$; para 1 A se obtiene $V_{\text{out}} = 2 \text{ V}$. Ambos valores quedan dentro del rango de entrada seleccionado para el sistema de adquisición.

3.1.2. Adquisición y promediado

La señal V_{out} se adquiere con un sistema autónomo compuesto por el convertidor analógico-digital ADS1115 [17] y la tarjeta de desarrollo LilyGO TTGO LoRa32 [14], basada en el microcontrolador ESP32. El ADS1115 digitaliza la salida del INA240 y la tarjeta de adquisición almacena las muestras en una microSD para su procesamiento posterior. La sincronización con la FPGA se realiza mediante un pulso digital generado por el firmware.

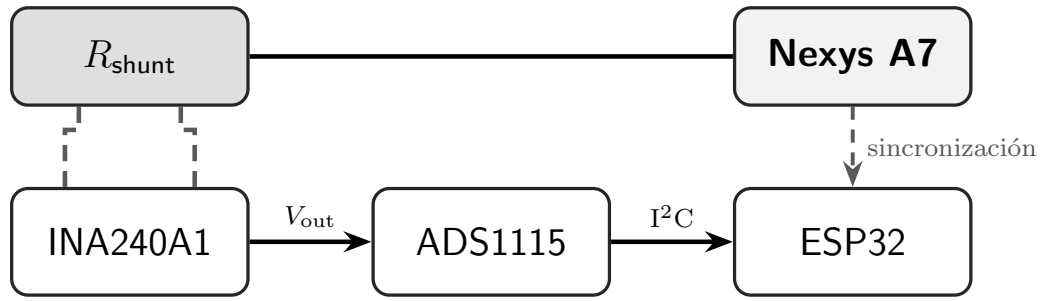


Figura 3.4: Cadena de adquisición de potencia desde el resistor de sensado hasta el sistema ESP32.

Se configura inicialmente $PGA = \pm 4,096 \text{ V}$ para evitar saturación durante las pruebas de corriente máxima. Para la caracterización fina se emplea $PGA = \pm 2,048 \text{ V}$, lo que proporciona una resolución de:

$$\Delta V = \frac{2 \times 2,048 \text{ V}}{2^{16}} \approx 62,5 \mu\text{V} \quad (3.6)$$

equivalente a $31,25 \mu\text{A}$ de resolución en corriente y $156 \mu\text{W}$ en potencia a través de la cadena de medición ($R_{\text{shunt}} = 100 \text{ m}\Omega$, $G = 20$). Esta configuración cubre corrientes de hasta 1 A ($V_{\text{out}} = 2 \text{ V}$), valor asociado al consumo máximo de placa reportado para una alimentación de 5 V [2].

La delimitación de la ventana de medición se realizará mediante una señal GPIO generada por el firmware. Al inicio de la región bajo análisis, el firmware activará la señal de sincronización; al final de la región la desactivará. La LilyGO ESP32 detectará ambos flancos para marcar el inicio y el final de las muestras, de modo que cada ejecución produzca un registro autocontenido asociado a la región de interés.

El promediado sobre N muestras reduce el ruido aleatorio en un factor $1/\sqrt{N}$ [18, 3]. Con una tasa de 860 SPS , las ventanas delimitadas por el pulso de sincronización proporcionan un número de muestras proporcional a la duración del segmento medido; para ventanas de 10 a 60 segundos se obtienen entre $8,6 \times 10^3$ y $5,2 \times 10^4$ muestras por ejecución, suficiente para estimar la potencia promedio de estado estacionario con baja dispersión. La potencia promedio se calcula a partir del promedio de las muestras del archivo correspondiente:

$$\bar{P} = V_{DD} \cdot \frac{\bar{V}_{\text{out}}}{G \cdot R_{\text{shunt}}} = 5 \cdot \frac{\bar{V}_{\text{out}}}{20 \cdot 0,1} = 2,5 \bar{V}_{\text{out}} \quad [\text{W}] \quad (3.7)$$

donde $\bar{V}_{\text{out}}$ es el promedio de las muestras consideradas en el intervalo de medición. La ecuación 3.7 asume $V_{DD} = 5 \text{ V}$ constante, despreciando la caída sobre el resistor de sensado. Para una corriente de 1 A esta caída es de 100 mV , equivalente a una sobrestimación del 2% en la potencia calculada. Como ejemplo, para 400 mA la sobrestimación se reduce al $0,8 \%$. Cuando el análisis requiere energía total, esta se obtiene multiplicando la potencia promedio por la duración del intervalo:

$$E = \bar{P} \Delta t \quad (3.8)$$

El intervalo Δt podrá obtenerse del reloj interno de la ESP32, usando los flancos del GPIO como marcas de inicio y fin, o de la diferencia del CSR estándar `mcycle` del CV32E40P entre el inicio y el final de la región medida. La segunda opción ofrece mayor precisión al estar referida al mismo reloj que ejecuta las instrucciones, por lo que se usará como referencia temporal en los análisis posteriores.

Esta medición externa se utilizará como referencia para caracterizar y validar el modelo sobre la plataforma FPGA. Una vez obtenidos los coeficientes, el flujo de análisis podrá estimar el consumo asociado a una ejecución a partir de los registros CSR, sin repetir la medición eléctrica para cada región de código analizada.

3.2. Módulo clasificador de instrucciones

El segundo componente de la solución será un módulo hardware nuevo integrado directamente en el procesador CV32E40P. Su función será observar las instrucciones que completen su ejecución en el pipeline, clasificarlas según su tipo y acumular el conteo en registros accesibles mediante CSR. La estimación no se calculará dentro del firmware; los conteos se leerán desde el flujo de prueba y análisis para combinarlos con los coeficientes energéticos caracterizados.

3.2.1. Concepto de clasificación

El pipeline del CV32E40P decodifica cada instrucción en la etapa ID antes de ejecutarla, determinando qué unidades funcionales activará. Esta información es suficiente para clasificar la instrucción en una de las seis categorías adoptadas en este trabajo: *Arithmetic*, *Logic*, *Memory*, *Branch*, *Jump* y *Floating point*. Las instrucciones de carga y almacenamiento se agrupan en *Memory* porque ambas activan la unidad de carga/almacenamiento, la ALU para el cálculo de dirección, el bus TCDM y el banco de registros.

El módulo observará las señales que el pipeline genera para una instrucción válidamente contabilizable, de forma que no se incluyan instrucciones anuladas por stalls, saltos o vaciados del pipeline. La Tabla 3.1 resume, para cada categoría, el origen de la señal de clasificación. Las categorías *Memory*, *Branch* y *Jump* se identifican a partir de las señales de evento ya generadas por el procesador para los contadores de rendimiento hardware: `mhpmevent_load`, `mhpmevent_store`, `mhpmevent_branch` y `mhpmevent_jump`. La categoría *Floating point* se identifica con la habilitación `apu_en`, asociada a la ruta APU/FPU del núcleo. Las categorías *Arithmetic* y *Logic* comparten la habilitación `alu_en`, por lo que el módulo las discrimina combinando `alu_en` con el código `alu_operator` generado por el decodificador: cuando la operación corresponde al subconjunto aritmético (sumas, restas, comparaciones, división y resto) se contabiliza como *Arithmetic*, y cuando corresponde

al subconjunto lógico (`and`, `or`, `xor`, desplazamientos y operaciones de manipulación de bits) se contabiliza como *Logic*. Las instrucciones de multiplicación, detectadas mediante `mult_en`, también se asignan a *Arithmetic*. En todos los casos la clasificación se habilita únicamente cuando la instrucción ha sido retirada, usando la señal `mhpmevent_minstret`. De este modo no se contabilizarán instrucciones anuladas ni intentos de ejecución que no llegan a completarse. Adicionalmente, el conteo se inhibirá cuando la instrucción retirada corresponda a un acceso CSR, identificado mediante la señal `csr_access_ex`: el decodificador del CV32E40P deja activa la ALU para `csrr`, `csrw`, `csrs` y `csrc`, lo que llevaría a contabilizar estas instrucciones como *Arithmetic*. Filtrarlas evitará que las propias lecturas y escrituras de los contadores contaminen la categoría aritmética durante la medición.

Tabla 3.1: Categoría de instrucción y origen de la señal usada para clasificarla en el pipeline del CV32E40P.

Categoría	Condición de detección	Detalle
Arithmetic	<code>mult_en</code> o <code>alu_en</code> con <code>alu_operator</code> aritmético	Multiplicaciones del multiplicador entero u operaciones aritméticas de la ALU (<code>add</code> , <code>sub</code> , <code>slt</code> , <code>mul</code> y <code>div</code>).
Logic	<code>alu_en</code> con <code>alu_operator</code> lógico	Subconjunto lógico del decodificador de la ALU (<code>and</code> , <code>or</code> , <code>xor</code> , etc).
Memory	<code>mhpmevent_load</code> o <code>mhpmevent_store</code>	Eventos de carga y almacenamiento ya generados por el core (<code>lw</code> , <code>sw</code> , etc.).
Branch	<code>mhpmevent_branch</code>	Evento de salto condicional retirado, generado por la lógica interna del core (<code>beq</code> , <code>bne</code> , etc).
Jump	<code>mhpmevent_jump</code>	Evento de salto incondicional retirado, correspondiente a <code>jal</code> / <code>jalr</code> .
Floating point	<code>apu_en</code>	Habilitación de la unidad de coprocesador (FPU/Xpulp).

3.2.2. Contadores e interfaz CSR

Por cada categoría existirá un contador de 64 bits que almacenará el conteo acumulado de instrucciones retiradas de ese tipo. Esta anchura evita desbordes durante ventanas de medición de decenas de segundos a 100 MHz, donde un contador de 32 bits podría saturarse en menos de un minuto si una categoría domina la ejecución. Cada vez que el pipeline complete una instrucción de una categoría, la lógica del módulo incrementará el contador correspondiente.

Como el CV32E40P es un procesador de 32 bits, cada contador de 64 bits se expone

mediante dos CSR personalizados de 32 bits: una palabra baja y una palabra alta. Estos CSR se ubicarán en el espacio de direcciones reservado por la especificación RISC-V para implementaciones de modo máquina [13]. El acceso usará las instrucciones estándar de la ISA, sin requerir sistema operativo ni llamadas al sistema. Las lecturas `csrr` permitirán reconstruir el conteo acumulado de cada categoría desde el flujo de prueba, mientras que escribir cero en ambos CSR reiniciará el contador correspondiente. De esta forma, una región específica de código podrá delimitarse sin reiniciar el procesador completo.

3.2.3. Integración en el procesador

El módulo se integrará como un bloque independiente dentro del núcleo CV32E40P, sin alterar la semántica de ejecución del pipeline. En el árbol RTL del procesador, el módulo se instanciará en `cv32e40p_core.sv` junto al resto de las unidades funcionales, y recibirá como entradas señales ya disponibles en el núcleo: `mhpmevent_minstret`, `mhpmevent_load`, `mhpmevent_store`, `mhpmevent_branch`, `mhpmevent_jump`, además de `alu_en`, `alu_operator`, `mult_en` y `apu_en`. Estas señales permitirán reutilizar la información interna del procesador sin introducir una nueva etapa de decodificación.

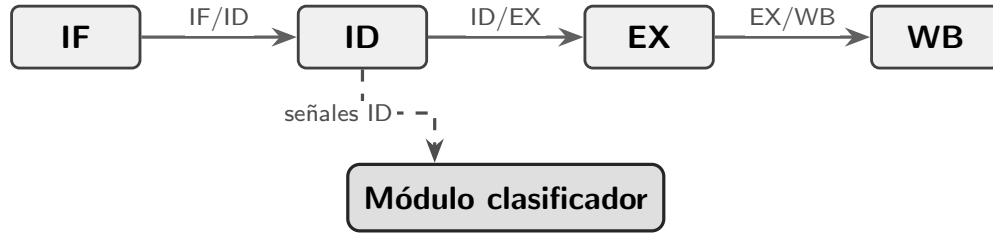


Figura 3.5: Integración conceptual del módulo clasificador como observador del pipeline del CV32E40P.

La integración con los CSR se realizará también en `cv32e40p_core.sv`. En lugar de modificar el bloque `cv32e40p_cs_registers.sv` para que almacene directamente los contadores del clasificador, se conservará su salida original en una señal intermedia y se añadirá un multiplexor de lectura. Cuando la dirección CSR solicitada pertenezca al clasificador, el núcleo devolverá el dato generado por este bloque; en caso contrario, devolverá el valor original del banco CSR estándar. Las direcciones usadas para los doce CSR del clasificador serán `0xBC0--0xBCB`, dentro del espacio reservado para implementaciones de modo máquina. Con este esquema, el clasificador se comportará como una extensión del espacio CSR sin modificar el funcionamiento de registros existentes como `mcycle` o `minstret`.

3.2.4. Carga del firmware vía JTAG

La carga del firmware al CV32E40P y la verificación del funcionamiento del módulo clasificador se realizarán mediante la interfaz de depuración JTAG integrada en el procesador [5]. La cadena de comunicación, ilustrada en la Figura 3.6, consta de tres elementos:

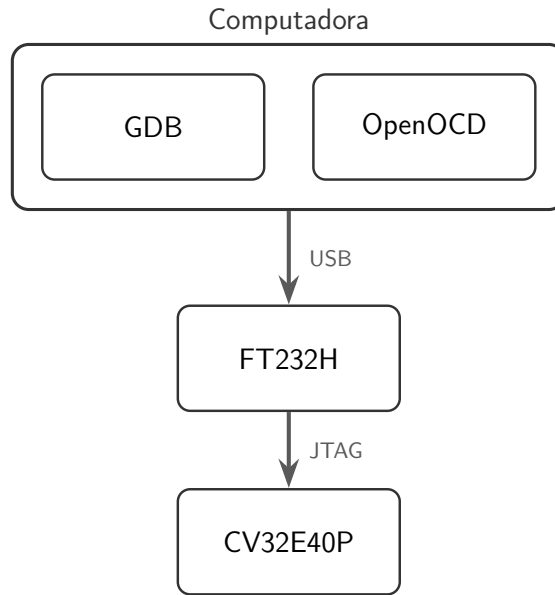


Figura 3.6: Cadena de comunicación con el CV32E40P.

El módulo FT232H [4] actúa como puente entre el puerto USB de la computadora y las señales JTAG del procesador mediante el motor MPSSE (*Multi-Protocol Synchronous Serial Engine*). Sobre esta interfaz, OpenOCD maneja el protocolo JTAG e implementa el acceso definido por la especificación RISC-V Debug [12, 10]. GDB se conecta a OpenOCD para cargar el binario en la SRAM del SoC, ejecutar el firmware y leer los registros CSR del clasificador durante la verificación.

El módulo físico empleado para esta interfaz se muestra en la Figura 3.7.



Figura 3.7: Módulo FT232H utilizado como interfaz USB-JTAG.

3.3. Caracterización y estimación

La caracterización de coeficientes energéticos se abordará mediante dos métodos complementarios. En el método de bucles dominados por categoría, cada programa de prueba se construirá para que una categoría represente al menos el 95 % de las instrucciones ejecutadas. En este caso no será necesario leer los CSR del clasificador para conocer la categoría dominante, porque el perfil de la rutina estará definido por construcción. La potencia promedio se obtendrá mediante una medición en régimen estable durante ventanas de 10 a 60 segundos.

El método de regresión por histograma requerirá que el circuito de medición (sección 3.1) y el módulo clasificador (sección 3.2) operen en conjunto. En ese caso, la plataforma producirá dos flujos de datos complementarios: las muestras de potencia registradas por el sistema ADS1115–LilyGO ESP32 en la microSD y los conteos de instrucciones por categoría leídos de los registros CSR del procesador. Ambos flujos se combinarán para obtener los coeficientes energéticos e_i del modelo de estimación, válidos para esta implementación en FPGA.

Ambos métodos producirán un conjunto de coeficientes e_i asociados a cada categoría de instrucción que posteriormente se compararán según el error de estimación obtenido frente a las mediciones físicas de referencia, de modo que la selección final del conjunto de coeficientes dependerá del desempeño experimental del modelo y no de una preferencia previa por un método particular.

3.3.1. Software de análisis y operación

El software de apoyo se organizará como un flujo externo de calibración, validación y operación. Su función será automatizar la ejecución de firmware en la plataforma, asociar cada ventana de ejecución con las muestras registradas por el sistema de adquisición y, cuando el clasificador esté integrado, leer los conteos acumulados en los CSR. A partir de estos datos se calcularán los coeficientes energéticos del modelo y se generarán los reportes necesarios para comparar la estimación contra la medición física.

Durante la calibración, el software procesará dos tipos de ejecuciones. En las rutinas dominadas por categoría, calculará la potencia promedio de cada ventana y derivará un coeficiente inicial para la categoría correspondiente. En las cargas mixtas, combinará la energía medida con los histogramas de instrucciones reportados por el clasificador y resolverá el sistema de regresión descrito en la sección 2.2.1. La validación posterior usará programas distintos a los de calibración para comparar la potencia promedio estimada con la potencia promedio medida y calcular el error relativo del modelo.

Una vez caracterizada la plataforma, el modo de operación final no requerirá la cadena de medición eléctrica. El usuario ejecutará un firmware sobre el sistema, el software leerá los conteos por categoría al finalizar la región de interés y aplicará los coeficientes

almacenados para reportar energía, potencia y desglose por categoría. De esta forma, la instrumentación externa se usará solo durante la fase de caracterización; la estimación posterior dependerá únicamente del clasificador integrado y del archivo de coeficientes obtenido experimentalmente.

Capítulo 4

Hipótesis y Objetivos

4.1. Hipótesis

Es posible estimar el consumo energético asociado a la ejecución de aplicaciones bare-metal sobre el procesador CV32E40P con un error relativo medio inferior al 10 % respecto a la medición eléctrica directa de la plataforma FPGA, mediante la integración de un módulo hardware de clasificación de instrucciones en el procesador y la aplicación de un modelo de estimación por categoría de instrucción caracterizado experimentalmente sobre la plataforma FPGA Nexys A7-100T.

El umbral del 10 % se adopta como criterio de desempeño para evaluar la utilidad del modelo en la comparación de regiones de código. Este margen toma en cuenta que la plataforma objetivo opera en bare-metal sobre FPGA, sin infraestructura de monitoreo energético del sistema operativo, y que cada categoría agrupa instrucciones con comportamientos eléctricos distintos. Bajo estas condiciones, el objetivo no es reemplazar la medición eléctrica de referencia durante la caracterización, sino obtener una estimación suficientemente precisa para orientar el análisis energético del firmware.

4.2. Objetivo general

Diseñar e implementar un módulo hardware de clasificación de instrucciones integrado en el procesador CV32E40P sobre la plataforma PULPissimo, que genere conteos por categoría para estimar, mediante un flujo externo de análisis, el consumo energético de aplicaciones bare-metal ejecutadas en la FPGA.

4.3. Objetivos específicos

1. Diseñar y construir el circuito de medición de potencia basado en resistor de sensado de corriente y amplificador de sensado INA240 en configuración *high-side*, y verificar su correcto funcionamiento sobre la línea de alimentación de la placa Nexys A7-100T.
2. Implementar en SystemVerilog el módulo clasificador de instrucciones como bloque RTL integrado en el núcleo CV32E40P, exponiendo un contador de 64 bits por cada una de las seis categorías de instrucción definidas mediante pares de CSR de 32 bits.
3. Sintetizar el SoC PULPissimo con el módulo clasificador incorporado sobre la FPGA Nexys A7-100T y verificar el funcionamiento de los registros CSR mediante firmware bare-metal y la cadena JTAG.
4. Caracterizar experimentalmente los coeficientes de energía e_i por categoría de instrucción sobre la plataforma FPGA, aplicando el método de bucles dominados por categoría y el método de regresión por histograma.
5. Desarrollar el pipeline de análisis de datos que automatice el procesamiento de las mediciones de potencia adquiridas por el sistema ADS1115–LilyGO ESP32 y la lectura de registros CSR, y genere estimaciones de energía, potencia promedio y desglose por categoría.
6. Validar el modelo de estimación comparando la potencia promedio estimada $\bar{P}_{\text{est}} = E_{\text{est}}/\Delta t$ contra la potencia promedio medida físicamente en la plataforma FPGA durante la ejecución de cargas de trabajo mixtas no utilizadas en la caracterización.

4.4. Tareas asociadas a los objetivos específicos

OE1: Circuito de medición de potencia.

- Establecer la topología de medición *high-side* sobre la línea de alimentación de la Nexys A7-100T.
- Dimensionar el resistor de sensado a partir del rango de corriente definido para la plataforma.
- Conectar y probar la cadena INA240A1–ADS1115–LilyGO ESP32.
- Calibrar la conversión de tensión medida a corriente y potencia.

OE2: Módulo clasificador de instrucciones.

- Establecer y justificar las categorías de instrucciones a contabilizar.
- Seleccionar y justificar las señales del pipeline del CV32E40P utilizadas para detectar instrucciones retiradas válidamente.
- Implementar la lógica de clasificación en SystemVerilog.
- Implementar los contadores de 64 bits por categoría y su acceso mediante pares de CSR de 32 bits.

OE3: Integración y verificación en FPGA.

- Integrar el clasificador con el banco de CSR del CV32E40P.
- Sintetizar el SoC PULPissimo modificado para la FPGA Nexys A7-100T.
- Cargar y ejecutar firmware bare-metal mediante la cadena GDB–OpenOCD–JTAG.
- Verificar la lectura y el reinicio de los contadores mediante acceso CSR.

OE4: Caracterización de coeficientes energéticos.

- Implementar rutinas dominadas por categoría para las categorías de instrucción.
- Medir la potencia promedio de cada categoría en régimen estable.
- Ejecutar cargas mixtas y recolectar histogramas de instrucciones.
- Calcular coeficientes mediante bucles dominados por categoría y regresión por histograma.

OE5: Pipeline de análisis de datos.

- Automatizar la lectura de mediciones adquiridas por el sistema ADS1115–LilyGO ESP32.
- Procesar las muestras de potencia y calcular energía por ejecución.
- Integrar la lectura de contadores CSR en el flujo de análisis.
- Generar estimaciones, tablas y visualizaciones por categoría.

OE6: Validación del modelo.

- Definir un conjunto de cargas mixtas no utilizadas durante la caracterización.
- Ejecutar las cargas sobre la plataforma FPGA y medir su potencia.
- Comparar la potencia promedio estimada contra la potencia promedio medida.
- Calcular el error relativo y verificar el cumplimiento de la hipótesis.

4.5. Alcances y limitaciones

El proyecto abarca el diseño, implementación y validación experimental de un módulo hardware de clasificación de instrucciones integrado en el procesador CV32E40P dentro del SoC PULPissimo. La implementación se realizará sobre la plataforma FPGA Nexys A7-100T y estará orientada a firmware bare-metal, sin dependencia de sistema operativo.

La estimación energética se limitará a un modelo por categoría de instrucción. Para ello se considerarán seis categorías principales: *Arithmetic*, *Logic*, *Memory*, *Branch*, *Jump* y *Floating point*. Cada categoría tendrá un contador de 64 bits expuesto mediante pares de CSR personalizados de 32 bits.

El trabajo incluye la construcción y uso de una cadena de medición de potencia externa basada en un sensor de corriente INA240A1, un convertidor ADS1115 y una tarjeta ESP32

para adquisición de datos. Esta cadena se utilizará como referencia física para caracterizar los coeficientes energéticos del modelo y validar la estimación calculada a partir de los contadores internos.

La caracterización experimental considerará tanto el método de bucles dominados por categoría como el método de regresión por histograma. Los coeficientes obtenidos por ambos métodos se compararán según su error frente a mediciones físicas de referencia, y el conjunto con mejor desempeño se utilizará para la validación final del modelo.

Como limitación principal, los coeficientes energéticos obtenidos serán válidos para la configuración experimental utilizada: CV32E40P integrado en PULPissimo, sintetizado sobre la FPGA Nexys A7-100T y operando bajo las condiciones de alimentación, frecuencia y bitstream definidas durante la caracterización. Por tanto, no se asume que los coeficientes puedan trasladarse directamente a un ASIC, a otra FPGA o a otra frecuencia de operación.

La medición eléctrica corresponde al consumo observado en la plataforma FPGA completa bajo la configuración experimental, no al consumo aislado del núcleo CV32E40P como circuito independiente. El modelo resultante estima el consumo asociado a una ejecución en esa plataforma, pero no separa explícitamente el aporte de todos los bloques internos del SoC ni de los reguladores de la placa.

El clasificador trabaja a nivel de categorías de instrucción, no a nivel de instrucciones individuales. En consecuencia, instrucciones distintas dentro de una misma categoría comparten un coeficiente energético promedio, por lo que el modelo puede perder precisión en programas donde una categoría agrupe operaciones con comportamientos eléctricos muy diferentes.

La validación se realizará con cargas bare-metal controladas y no contempla ejecución bajo sistema operativo, multiprogramación, interrupciones complejas ni periféricos externos con patrones de actividad no controlados. Además, el modelo no pretende reemplazar la medición eléctrica durante la caracterización inicial; la instrumentación externa sigue siendo necesaria para obtener los coeficientes y validar la hipótesis.

Capítulo 5

Experimento preliminar

El experimento preliminar valida el flujo de medición antes de la caracterización completa. Usa rutinas dominadas por una categoría conocida para comprobar que la plataforma muestra diferencias de potencia.

Reutiliza la cadena JTAG y la cadena de medición de potencia ya descritas, sin introducir una arquitectura de prueba adicional.

5.1. Objetivo del experimento

Verificar que rutinas dominadas por distintas categorías producen diferencias medibles y repetibles en la potencia promedio.

5.2. Configuración de la prueba

El SoC PULPissimo se sintetizará sobre la FPGA Nexys A7-100T y ejecutará firmware bare-metal cargado mediante GDB, OpenOCD y FT232H/JTAG. La corriente se medirá con el INA240A1 y el ADS1115–LilyGO ESP32.

Cada rutina se ejecutará durante 10 a 30 segundos y el firmware marcará la ventana de medición con un pulso GPIO. La potencia promedio se calculará con la ecuación [3.7](#).

5.3. Rutinas de bucles dominados por categoría

Las rutinas de prueba ejecutan un bloque reducido de instrucciones de una sola categoría. El objetivo es que esa categoría domine la ejecución, aunque existan instrucciones auxiliares de control.

5.4. Datos a recolectar

Para cada rutina se registrarán la categoría dominante, el tiempo de adquisición, la potencia promedio y la dispersión entre repeticiones.

5.5. Criterio de viabilidad

El experimento será útil si rutinas de categorías distintas producen diferencias de potencia mayores que la variabilidad entre repeticiones. Si eso ocurre, queda justificada la caracterización posterior.

Capítulo 6

Cronograma

El cronograma cubre 17 semanas y sigue una secuencia simple: preparación, medición, implementación, análisis, validación y cierre. Las primeras semanas se usan para preparar el entorno, validar la cadena JTAG y comprobar la medición de potencia.

Después se ejecuta el experimento preliminar con bucles dominados por categoría. Esa actividad no depende todavía del clasificador RTL, así que puede hacerse pronto.

La etapa central corresponde al diseño del clasificador, su integración con los CSR del CV32E40P y la síntesis del SoC PULPissimo modificado sobre la FPGA Nexys A7-100T. Luego se hace la caracterización por regresión de histogramas y se construye el flujo de análisis.

Las semanas finales se reservan para validar el modelo con cargas mixtas, comparar la estimación contra la medición física de referencia y cerrar la documentación. La semana 17 queda como margen ante atrasos de síntesis o temporización.

Bibliografía

- [1] Peter Barker and Mohammad Hammoudeh. A survey on low power network protocols for the internet of things and wireless sensor networks. In *International Conference on Future Networks*, 2017.
- [2] Digilent Inc. *Nexys A7 FPGA Board Reference Manual*. Digilent Inc., 2019. URL https://digilent.com/reference/_media/reference/programmable-logic/nexys-a7/nexys-a7_rm.pdf. Nexys A7-100T: XC7A100T-1CSG324C, 15,850 slices, 126,800 FFs, 4,860 Kb BRAM, 240 DSP slices.
- [3] Gloria Yu Liang Fang. Instruction-level power consumption simulator for modeling simple timing and power side channels in a 32-bit RISC-V micro-processor. Master of engineering thesis, Massachusetts Institute of Technology, feb 2021. Supervisor: Prof. Anantha Chandrakasan.
- [4] Future Technology Devices International Ltd. *FT232H Single Channel Hi-Speed USB to Multipurpose UART/FIFO IC Datasheet*. FTDI Chip, 2014. URL <https://ftdichip.com/products/ft232h/>.
- [5] IEEE. IEEE standard for test access port and boundary-scan architecture, IEEE std 1149.1-2013. IEEE, 2013. DOI: 10.1109/IEEESTD.2013.6515989.
- [6] IEEE Computer Society. *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*. IEEE Std 1800-2012. IEEE, 2013. Revision of IEEE Std 1800-2009.
- [7] Miguel Jiménez Arribas, Agustín Martínez Hellín, Manuel Prieto Mateo, Iván Gaminó del Río, Andrea Fernández Gallego, Óscar Rodríguez Polo, Antonio da Silva, Pablo Parra, and Sebastián Sánchez. Design and implementation of a synchronous hardware performance monitor for a RISC-V space-oriented processor. *Microprocessors and Microsystems*, 2024. arXiv:2404.05389.
- [8] Luis G. León-Vega, Niccolò Tosato, and Stefano Cozzini. A comprehensive analysis of process energy consumption on multi-socket systems with GPUs. In *Latin American High Performance Computing Conference (CARLA)*, 2024. arXiv:2409.04941.

- [9] Luis G. León-Vega, Niccolò Tosato, and Stefano Cozzini. EfiMon: A process analyser for granular power consumption prediction. In *Latin American High Performance Computing Conference (CARLA)*, 2024. arXiv:2409.17368.
- [10] Tim Newsome and Paul Donahue. The RISC-V debug specification. Technical report, RISC-V International, 2024. URL <https://github.com/riscv/riscv-debug-spec>.
- [11] OpenHW Group. CV32E40P user manual. https://docs.openhwgroup.org/projects/cv32e40p-user-manual/en/cv32e40p_v1.8.3/, 2024. Version cv32e40p_v1.8.3.
- [12] OpenOCD Project. *Open On-Chip Debugger User's Guide*. OpenOCD Project, 2026. URL <https://openocd.org/doc/html/>.
- [13] RISC-V International. The RISC-V instruction set manual, volume I: Unprivileged architecture. Technical report, RISC-V International, 2026. Version 20260120, official release.
- [14] Shenzhen Xinyuan Electronic Technology Co., Ltd. (LilyGO). LilyGO TTGO LoRa32 development board. <https://github.com/Xinyuan-LilyGO/LilyGo-LoRa-Series>, 2023.
- [15] Satyajit Sinha. State of IoT 2025: Number of connected IoT devices growing 14 % to 21.1 billion globally. <https://iot-analytics.com/number-connected-iot-devices/>, 2025.
- [16] Texas Instruments. *INA240 High- and Low-Side, Bidirectional, Zero-Drift Current-Sense Amplifier*. Texas Instruments. URL <https://www.ti.com/lit/ds/symlink/ina240.pdf>.
- [17] Texas Instruments. *ADS1113/4/5 Ultra-Small, Low-Power, I²C-Compatible, 860-SPS, 16-Bit ADC with Internal Reference, Oscillator, and Programmable Comparator*. Texas Instruments, 2018. URL <https://www.ti.com/lit/ds/symlink/ads1115.pdf>.
- [18] Texas Instruments. An engineer's guide to current sensing. <https://www.ti.com/lit/eb/slyy154b/slyy154b.pdf>, 2023.
- [19] The PULP Team. PULPissimo: Datasheet. <https://github.com/pulp-platform/pulpissimo>, mar 2021. March 23, 2021.
- [20] Vivek Tiwari, Sharad Malik, and Andrew Wolfe. Power analysis of embedded software: A first step towards software power minimization. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2(4):437–445, dec 1994.